

คู่มืออธิบายโค้ด

Mini Job Scheduler & Shared Resource Manager

เอกสารนี้อธิบายไฟล์ .java ทั้ง 8 ไฟล์ที่เขียนขึ้นเองสำหรับโครงงาน (Job, ResourceManager, ReadyQueue, Statistics, JobGenerator, Scheduler, Worker, Monitor) ที่ละส่วนเป็นภาษาไทย เพื่อให้อธิบายต่อใน Demo ได้ด้วยตัวเอง

สารบัญ

1. ภาพรวมระบบและลำดับการไหลของงาน
2. คำตอบ: priority เท่ากัน และ arrival time เท่ากัน จัดการอย่างไร
3. Job.java
4. ResourceManager.java
5. ReadyQueue.java
6. Statistics.java
7. JobGenerator.java
8. Scheduler.java
9. Worker.java
10. Monitor.java

1. ภาพรวมระบบและลำดับการไหลของงาน

งานหนึ่งชิ้น (Job) เดินทางผ่านระบบตามลำดับนี้:

```
JobGenerator --(arrivalQueue, BlockingQueue<Job>)--> Scheduler
Scheduler --(readyQueue.add)-----> ReadyQueue
ReadyQueue --(readyQueue.take, บล็อกถ้าว่าง)-----> Worker (หลายตัว)
Worker --(resources.acquire/release)-----> ResourceManager (Semaphore)
Worker --(statistics.recordCompletion)-----> Statistics
Monitor --(อ่านทุกตัวด้านบนทุก ~1000ms)-----> log สถานะระบบ
```

มี "คิว" สองชั้น เพราะโจทย์บังคับว่า JobGenerator ห้ามใส่งานลง ReadyQueue ตรง ๆ ต้องผ่าน Scheduler เสมอ (arrivalQueue คือคิวชั้นแรก ระหว่าง JobGenerator กับ Scheduler, ReadyQueue คือคิวชั้นสองระหว่าง Scheduler กับ Worker)

การปิดระบบ (graceful shutdown) ใช้สองกลไกที่ทำหน้าที่ต่างกัน:

- POISON_PILL — Job ปลอมตัวหนึ่งที่ใช้บอกว่า "ไม่มีงานใหม่เข้ามาอีกแล้ว" ส่งต่อกันเป็นทอด ๆ: JobGenerator ส่งเข้า arrivalQueue 1 ตัว → Scheduler เห็นแล้ว ส่งเข้า ReadyQueue ให้ครบ "หนึ่งตัวต่อ Worker หนึ่งตัว" → Worker แต่ละตัวที่หยิบได้ pill ก็หยุดตัวเอง
- CountdownLatch — ตัวนับที่ Main สร้างด้วยขนาด = จำนวน Job ทั้งหมด ทุกครั้งที่ Worker ทำ Job จริงเสร็จ (ไม่ใช่ pill) จะเรียก countDown() หนึ่งครั้ง Main รอที่ latch.await() เพื่อรู้ว่า "งานชิ้นสุดท้ายเสร็จจริงแล้ว" ก่อนจะสั่งปิดระบบ

ทำไมต้องใช้สองกลไกแยกกัน: POISON_PILL บอกแค่ "จะไม่มีการใหม่เข้ามาอีก" แต่ตอนที่ส่ง pill ออกไป อาจยังมี Job ที่ Worker กำลังทำค้างอยู่ (ยังไม่เสร็จ) ถ้าใช้ pill อย่างเดียวตัดสินใจเลิกโปรแกรม อาจเลิกเร็วเกินไปจนงานสุดท้ายทำไม่เสร็จ จึงต้องมี latch แยกไว้ตรวจว่า "งานที่มีอยู่ทั้งหมดเสร็จจริงหรือยัง"

2. คำตอบ: priority เท่ากัน และ arrival time เท่ากัน จัดการอย่างไร

ตัวตัดสินลำดับ (tie-break) ที่ใช้จริงในโค้ดอยู่ใน ReadyQueue.java ตรง Comparator ที่สร้างตอน policy เป็น PRIORITY:

```
Comparator<Job> byPriorityThenSequence =  
    Comparator.<Job>comparingInt(j -> j.priority)  
        .thenComparingInt(j -> j.sequence);
```

ลำดับการเทียบมี 2 ชั้น:

- ชั้นที่ 1: เทียบ priority ก่อน (1 = สำคัญที่สุด) ถ้าต่างกัน จบเลย ไม่ต้องดูต่อ
- ชั้นที่ 2: ถ้า priority เท่ากัน ให้เทียบ sequence ต่อ (เลขลำดับที่ปรากฏในไฟล์ workload เริ่มจาก 0 — WorkloadLoader เป็นคนกำหนดให้ตอนโหลดไฟล์ แต่ละ Job ได้เลขไม่ซ้ำกันเลย)

กรณีที่ถามคือ priority เท่ากัน และ arrivalMs ก็เท่ากัน ด้วย — ทำอย่างไร?

คำตอบ: โค้ดนี้ไม่ได้ใช้ arrivalMs เป็นตัวตัดสินลำดับที่สองเลย ใช้ sequence แทน ซึ่งแก้ปัญหานี้ได้เบ็ดเสร็จ เพราะ sequence ไม่มีทางซ้ำกัน ระหว่าง Job สองตัวได้เลย (ต่างจาก arrivalMs ที่สองบรรทัดในไฟล์ CSV ตั้งค่าให้เท่ากันได้ ตามใจผู้เขียนไฟล์ workload)

ตัวอย่าง: สมมติไฟล์ workload มี 2 งานที่ priority = 5 และ arrivalMs = 100 เท่ากันทั้งคู่ แต่ตัวหนึ่งอยู่บรรทัดที่ 3 ของไฟล์ (sequence = 2) อีกตัวอยู่บรรทัดที่ 7 (sequence = 6) — เมื่อ priority เสมอกัน comparator จะไปเทียบ sequence ต่อทันทีที่ 2 < 6 จึงได้ว่า Job ที่ sequence = 2 (อยู่บรรทัดก่อนในไฟล์) ถูกเลือกก่อนเสมอ ไม่ว่าจะรันโปรแกรมกี่ครั้งก็ตาม

เพราะ sequence เป็นเลขที่ไม่ซ้ำกันเลยสักตัวในหนึ่งไฟล์ workload (0, 1, 2, 3, ...) การเทียบ priority แล้วตามด้วย sequence จึงทำให้ "ไม่มีทางเกิดการเสมอกันจริง ๆ" ได้อีกต่อไป (total order) ไม่ว่า priority จะเท่ากันกี่ตัว หรือ arrivalMs จะเท่ากันแค่ไหน ผลลัพธ์สุดท้ายจะตัดสินได้เสมอด้วย sequence ซึ่งเป็นข้อมูลของ Job เอง ไม่ได้ขึ้นกับว่า Thread ไหนมาถึงคิวก่อน — ตรงตามข้อบังคับหัวข้อ 4 ของโจทย์ที่ว่า "กติกาตัดสินลำดับต้องไม่ขึ้นกับ จังหวะของ Thread" พอดี

ทำไมไม่ใช้ arrivalMs เป็นตัวตัดสินที่สอง (ก่อนใช้ sequence)? เพราะ arrivalMs ก็ยังเสมอกันได้ เหมือนที่โจทย์ถาม ถ้าจะใช้ arrivalMs ต้องมีตัวตัดสินชั้นที่ 3 ต่ออีกอยู่ดี ในเมื่อ sequence ตัดสินได้เบ็ดเสร็จในชั้นเดียว (ไม่มีทางเสมอ) จึงเลือกใช้ sequence แทนตั้งแต่ชั้นที่สอง ไม่ต้องมีชั้นที่สาม

3. Job.java

เก็บข้อมูลของงานหนึ่งชิ้น แบ่งเป็น 2 กลุ่ม: 필드 final ที่มาจากไฟล์ CSV โดยตรง (กลุ่มที่อาจารย์แจกให้) กับ 필드 private ที่เพิ่มเองเพื่อเก็บเวลาแต่ละช่วงของ Job สำหรับเอาไปคำนวณ Waiting Time / Turnaround Time ที่หลัง

```
/**
 * ข้อมูลของงานหนึ่งชิ้น
 *
 * 필드ทั้งหมดในไฟล์นี้มาจากไฟล์ workload CSV โดยตรง และถูกกำหนดครั้งเดียว
 * ตอนโหลด จึงประกาศเป็น final และปลอดภัยเมื่อหลาย Thread อ่านพร้อมกัน
 *
 * ไฟล์นี้เป็นโค้ดตั้งต้นที่อาจารย์แจก แต่ต่างจากไฟล์อินตรงที่
 * นักศึกษา "ต้องแก้" โดยเพิ่มฟิลด์ของตัวเองในส่วน TODO ด้านล่าง
 */
public class Job {

    /** รหัสงาน เช่น J01 — ไม่ซ้ำกันภายในหนึ่งไฟล์ workload */
    public final String id;

    /** เวลาที่งานควรเข้าสู่ระบบ นับจากวินาทีที่โปรแกรมเริ่ม (มิลลิวินาที) */
    public final long arrivalMs;

    /** ระดับความสำคัญ โดย 1 คือสูงสุด ตัวเลขยิ่งมากยิ่งสำคัญน้อย */
    public final int priority;

    /** ระยะเวลาของงานหลัก ก่อนขอใช้ทรัพยากรร่วม (มิลลิวินาที) */
    public final long workMs;

    /** ทรัพยากรร่วมที่ต้องใช้ หรือ NONE ถ้าไม่ต้องใช้ */
    public final ResourceType resource;

    /** ระยะเวลาที่ถือครองทรัพยากร (มิลลิวินาที) เป็น 0 เสมอเมื่อ resource เป็น NONE */
    public final long resourceMs;

    /**
     * ลำดับที่งานนี้ปรากฏในไฟล์ workload เริ่มจาก 0
     * ใช้เป็นตัวตัดสินลำดับ (tie-break) เมื่อ priority เท่ากัน
     * เพราะเลขนี้ไม่ซ้ำกันเลยในแต่ละ Job (WorkloadLoader กำหนดให้ตอนโหลด)
     */
    public final int sequence;

    public Job(String id, long arrivalMs, int priority, long workMs,
               ResourceType resource, long resourceMs, int sequence) {
        this.id = id;
        this.arrivalMs = arrivalMs;
        this.priority = priority;
        this.workMs = workMs;
        this.resource = resource;
        this.resourceMs = resourceMs;
        this.sequence = sequence;
    }

    // =====
    // 필드ด้านล่างนี้เพิ่มเข้ามาเองเพื่อเก็บเวลาที่ใช้คำนวณผล (หัวข้อ 8)
    // ทั้งหมดอ่านจาก ProjectLogger.now() เพื่อให้เป็นนาฬิกาเดียวกับ log
    //
    // เป็น private + เข้าถึงผ่าน get/set ที่เป็น synchronized เพราะฟิลด์พวกนี้
    // เขียนโดย Thread หนึ่ง (JobGenerator หรือ Worker) แล้วอ่านโดยอีก Thread
    // หนึ่ง (Main/Statistics ตอนสรุปผล) synchronized ทำให้การอ่านเห็นค่าล่าสุด
    // เสมอ ไม่ว่าจะอ่านตอนไหนก็ตาม
    // =====

    private long actualArrivalMs;
    private long startMs;
    private long completionMs;
    private long resourceWaitStartMs;
    private long resourceWaitMs;

    public synchronized void setActualArrivalMs(long t) { this.actualArrivalMs = t; }
    public synchronized long getActualArrivalMs() { return actualArrivalMs; }
```

```

public synchronized void setStartMs(long t) { this.startMs = t; }
public synchronized long getStartMs() { return startMs; }

public synchronized void setCompletionMs(long t) { this.completionMs = t; }
public synchronized long getCompletionMs() { return completionMs; }

public synchronized void setResourceWaitStartMs(long t) { this.resourceWaitStartMs = t; }
public synchronized long getResourceWaitStartMs() { return resourceWaitStartMs; }

public synchronized void setResourceWaitMs(long t) { this.resourceWaitMs = t; }
public synchronized long getResourceWaitMs() { return resourceWaitMs; }

/** Waiting Time = startMs - actualArrivalMs */
public synchronized long waitingTimeMs() {
    return startMs - actualArrivalMs;
}

/** Turnaround Time = completionMs - actualArrivalMs */
public synchronized long turnaroundTimeMs() {
    return completionMs - actualArrivalMs;
}

@Override
public String toString() {
    return String.format("%s(priority=%d, work=%dms, %s)",
        id, priority, workMs,
        resource == ResourceType.NONE ? "no resource"
        : resource + " " + resourceMs + "ms");
}
}

```

คำอธิบายทีละส่วน

- 필드 final 7 ตัว (id, arrivalMs, priority, workMs, resource, resourceMs, sequence) — ถูกกำหนดค่าครั้งเดียวตอนสร้าง object (ใน constructor) แล้วห้ามแก้อีกเลยตลอดชีวิตของ Job เพราะเป็น final ข้อดีคือหลาย Thread อ่านพร้อมกันได้อย่างปลอดภัยโดยไม่ต้องป้องกันอะไรเลย (เพราะมันไม่มีวันเปลี่ยนค่า)
- sequence คือเลขลำดับที่ Job นี้ปรากฏในไฟล์ (เริ่มจาก 0) ใช้เป็นตัวตัดสินลำดับ (tie-break) เมื่อ priority เท่ากัน ใน ReadyQueue.java (ดูหัวข้อ 2)
- constructor — รับค่าพารามิเตอร์ 7 ตัว แล้วก๊อปปี้ทีละตัวใส่ this.field ตรง ๆ ไม่มีการคำนวณอะไรเพิ่ม
- 필드 private 5 ตัว (actualArrivalMs, startMs, completionMs, resourceWaitStartMs, resourceWaitMs) — เป็นเวลาที่ "ยังไม่รู้ค่าตอนสร้าง Job" ต้องรอ JobGenerator/Worker มาเขียนทีหลังตอนที่เหตุการณ์นั้นเกิดขึ้นจริง จึงเป็น private (ไม่ final) และมี get/set แยก
- ทำไมแต่ละ get/set ถึงมีคำว่า synchronized — 필드พวกนี้ถูกเขียนโดย Thread หนึ่ง (เช่น Worker ตอนเริ่มงาน) แล้วถูกอ่านโดยอีก Thread หนึ่ง (เช่น Main ตอนสรุปผล) synchronized ทำให้ Thread ที่อ่านมองเห็นค่าล่าสุดที่ Thread อื่นเขียนไว้เสมอ ถ้าไม่ใส่ไว้ อาจเกิดปัญหาที่ Thread หนึ่งเขียนค่าไปแล้ว แต่อีก Thread หนึ่งยังอ่านเห็นค่าเก่าอยู่ (ปัญหาเรื่อง memory visibility ซึ่งเป็นเรื่องปกติของโปรแกรมหลาย Thread)
- waitingTimeMs() — คำนวณค่า startMs - actualArrivalMs ตรงตามสูตรหัวข้อ 8 ของโจทย์
- turnaroundTimeMs() — คำนวณค่า completionMs - actualArrivalMs ตรงตามสูตรหัวข้อ 8 เช่นกัน
- toString() — แค่อ่าน debug/print เฉย ๆ ใช้ตัวดำเนินการ ?: (ternary) เพื่อเลือกข้อความว่าจะโชว์ว่า "no resource" หรือโชว์ชื่อ resource พร้อมเวลา ขึ้นกับว่า resource เป็น NONE หรือไม่

4. ResourceManager.java

ควบคุมสิทธิ์การใช้ PRINTER และ DATABASE ด้วย Semaphore ซึ่งเป็นคลาสสำเร็จรูปของ Java ที่ทำหน้าที่ "นับจำนวนสิทธิ์ที่เหลือ" ให้อัตโนมัติ และทำให้ Thread ที่ขอสิทธิ์ตอนที่เหลือแล้ว ต้องรอ (block) โดยไม่กิน CPU

```
import java.util.concurrent.Semaphore;

/**
 * ควบคุมสิทธิ์การใช้ทรัพยากรร่วมของทั้งระบบ
 *
 * ใช้ Semaphore สองตัวแยกกัน คนละ resource (PRINTER, DATABASE)
 * ต้องสร้างแบบ fair = true (โจทยบังคับ) เพื่อให้ Thread ที่รอมานาน
 * ได้สิทธิ์ก่อนเสมอ ไม่ใช่ใครมาใหม่ได้ก่อน (ป้องกัน starvation)
 */
public class ResourceManager {

    private final Semaphore printerSemaphore;
    private final Semaphore databaseSemaphore;

    private final int printerPermits;
    private final int databasePermits;

    public ResourceManager(int printerPermits, int databasePermits) {
        this.printerPermits = printerPermits;
        this.databasePermits = databasePermits;
        this.printerSemaphore = new Semaphore(printerPermits, true);
        this.databaseSemaphore = new Semaphore(databasePermits, true);
    }

    /** ขอสิทธิ์ใช้ทรัพยากร จะรอจนกว่าจะได้ */
    public void acquire(ResourceType type) throws InterruptedException {
        semaphoreFor(type).acquire();
    }

    /** คืนสิทธิ์ใช้ทรัพยากร ต้องเรียกใน finally เสมอ เพื่อไม่ให้ permit ค้าง */
    public void release(ResourceType type) {
        semaphoreFor(type).release();
    }

    /**
     * ข้อความสั้น ๆ บอกสถานะการใช้ทรัพยากร สำหรับส่งให้ ProjectLogger.monitor()
     * เช่น "printer=1/1 database=0/2"
     */
    public String status() {
        int printerInUse = printerPermits - printerSemaphore.availablePermits();
        int databaseInUse = databasePermits - databaseSemaphore.availablePermits();
        return String.format("printer=%d/%d database=%d/%d",
            printerInUse, printerPermits, databaseInUse, databasePermits);
    }

    private Semaphore semaphoreFor(ResourceType type) {
        if (type == ResourceType.PRINTER) {
            return printerSemaphore;
        } else if (type == ResourceType.DATABASE) {
            return databaseSemaphore;
        }
        throw new IllegalArgumentException("ไม่รองรับ resource: " + type);
    }
}
```

คำอธิบายทีละส่วน

- printerSemaphore, databaseSemaphore — Semaphore คนละตัวสำหรับคนละ resource เพื่อไม่ให้เกิดการแย่ง PRINTER ไปกระทบการแย่ง DATABASE
- printerPermits, databasePermits — เก็บจำนวน permit ทั้งหมดไว้เฉย ๆ เพื่อใช้คำนวณใน status() (Semaphore บอกแค่ "เหลือเท่าไร" ผ่าน availablePermits() ไม่ได้บอก "ใช้ไปเท่าไร" ตรง ๆ)

- constructor — สร้าง new Semaphore(permits, true) พารามิเตอร์ตัวที่สอง (true) คือ fair mode ซึ่งโจทย์บังคับให้ใช้หมายความว่า Thread ที่มาขอก่อนจะได้สิทธิ์ก่อนเสมอ (เรียงคิว FIFO) ถ้าใช้ false (ไม่ fair) ลำดับการได้สิทธิ์จะไม่แน่นอน ทำให้บาง Thread อาจรอนานผิดปกติ (starvation)
- acquire(type) — เรียก semaphoreFor(type).acquire() ซึ่งจะ "รอ" (block) ถ้า permit หมด และลดจำนวน permit ลง 1 ทันทีที่ได้สิทธิ์
- release(type) — เพิ่มจำนวน permit กลับขึ้น 1 ผู้เรียกต้องเรียกใน finally เสมอ เพื่อไม่ให้ permit หายถาวรหากเกิด exception ระหว่างถือครองอยู่
- status() — คำนวณจำนวนที่ "กำลังถูกใช้อยู่" = permit ทั้งหมด - availablePermits() แล้วจัดรูปแบบข้อความด้วย String.format ให้ตรงกับที่ ProjectLogger.monitor() ต้องการ
- semaphoreFor(type) — เมธอดช่วยส่วนตัว (private) เลือกที่จะคืน Semaphore ตัวไหนตาม ResourceType ที่ส่งเข้ามา ถ้าส่ง NONE เข้ามา (ซึ่งไม่ควรเกิดขึ้น เพราะ Worker ต้องเช็คก่อนเรียก) จะโยน IllegalArgumentException ทั้งไว้เป็นการป้องกันข้อผิดพลาด (defensive programming)

5. ReadyQueue.java

คิวที่เก็บ Job ที่พร้อมให้ Worker หยิบไปทำ เลือกใช้โครงสร้างข้อมูลคนละแบบตามนโยบาย (FCFS หรือ Priority) โดยซ่อนรายละเอียดไว้หลัง interface เดียวกันคือ BlockingQueue

```
import java.util.Comparator;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.PriorityBlockingQueue;

/**
 * คิวงานที่พร้อมถูกหยิบไปทำ
 *
 * ใช้ BlockingQueue ของ Java เป็นแกนหลัก เพราะ take() ของมันพัก Thread เอง
 * เมื่อคิวว่าง (ไม่ busy wait) และ add()/take() ปลดล็อกเมื่อหลาย Thread
 * เรียกพร้อมกันอยู่แล้วโดยไม่ต้องเขียน synchronized เอง
 *
 * เลือกชนิดคิวตามนโยบาย:
 * - FCFS -> LinkedBlockingQueue (เข้าก่อนออกก่อน)
 * - PRIORITY -> PriorityBlockingQueue เรียงตาม priority (1 สำคัญสุด)
 *   ถ้า priority เท่ากัน ใช้ sequence (ลำดับในไฟล์) ตัดสิน
 *   sequence ไม่ซ้ำกันเลย จึงไม่มีทางเสมอกันสองชิ้นพร้อมกัน
 *
 * การหยุดระบบ: เมื่อไม่มีงานเหลือแล้ว Scheduler จะใส่ POISON_PILL ลงคิว
 * หนึ่งตัวต่อ Worker หนึ่งตัว Worker ที่หยิบได้ POISON_PILL ต้องหยุดทำงาน
 * แทนที่จะเอาไปประมวลผล (เช็คด้วย isPoisonPill())
 */
public class ReadyQueue {

    /** สัญญาณบอก Worker ว่า "ไม่มีงานให้ทำอีกแล้ว ให้หยุด" */
    public static final Job POISON_PILL =
        new Job("POISON_PILL", 0, Integer.MAX_VALUE, 0,
            ResourceType.NONE, 0, Integer.MAX_VALUE);

    public static boolean isPoisonPill(Job job) {
        return job == POISON_PILL;
    }

    private final BlockingQueue<Job> queue;

    public ReadyQueue(Config.Policy policy) {
        if (policy == Config.Policy.PRIORITY) {
            Comparator<Job> byPriorityThenSequence =
                Comparator.<Job>comparingInt(j -> j.priority)
                    .thenComparingInt(j -> j.sequence);
            this.queue = new PriorityBlockingQueue<>(11, byPriorityThenSequence);
        } else {
            this.queue = new LinkedBlockingQueue<>();
        }
    }

    /** ใส่งานเข้าคิว เรียกโดย Scheduler Thread */
    public void add(Job job) {
        queue.add(job);
    }

    /**
     * หยิบงานถัดไปตามนโยบาย เรียกโดย Worker Thread
     * ถ้ายังไม่มีงาน take() จะรอโดยไม่กิน CPU
     */
    public Job take() throws InterruptedException {
        return queue.take();
    }

    /** จำนวนงานที่รออยู่ตอนนี้ ใช้โดย Monitor */
    public int size() {
        return queue.size();
    }
}
```

คำอธิบายทีละส่วน

- POISON_PILL — Job ปลอมตัวหนึ่งทีสร้างด้วย priority และ sequence เป็น Integer.MAX_VALUE (ค่ามากที่สุดที่ int เก็บได้) เพื่อให้แน่ใจว่าต่อให้อยู่ใน Priority Queue มันจะถูกหยิบ "เป็นตัวสุดท้ายเสมอ" หลังงานจริงทุกตัว
- isPoisonPill(job) — เช็คด้วย == (เทียบว่าเป็น object เดียวกันปะะ ๆ ไม่ใช่แค่ค่าเหมือนกัน) ว่า Job ที่ได้มาคือ POISON_PILL ตัวนี้หรือไม่
- queue (field) — ประกาศเป็น type interface BlockingQueue<Job> ไม่ใช่ type ของคลาสจริง เพื่อให้เปลี่ยน implementation ได้อิสระในบรรทัดเดียวที่ constructor
- constructor — ถ้า policy เป็น PRIORITY: สร้าง Comparator ที่เทียบ priority ก่อน แล้ว ตามด้วย sequence (ด้วย .thenComparingInt) แล้วเอาไปสร้าง PriorityBlockingQueue (เลข 11 คือ "ความจุเริ่มต้น" เฉย ๆ ไม่ใช่ขีดจำกัด คิวขยายขนาดได้เองอัตโนมัติ) ถ้าเป็น FCFS: ใช้ LinkedBlockingQueue ธรรมดาซึ่งเก็บแบบเข้าก่อนออกก่อน (FIFO) อยู่แล้ว
- add(job) — เรียก queue.add(job) ใส่งานเข้าคิว เพราะคิวทั้งสองแบบนี้ไม่มีขีดจำกัดขนาด (unbounded) จึง add() ไม่มีวัน block
- take() — เรียก queue.take() ซึ่งเป็นหัวใจของการ "ไม่ busy wait": ถ้าคิวว่าง Thread ที่เรียกจะถูกพักไว้ (ไม่กิน CPU เลย) จนกว่าจะมีงานใหม่เข้ามา
- size() — คำนวณงานที่ค้างอยู่ในคิวตอนนี้ให้ Monitor ใช้รายงานสถานะ

6. Statistics.java

เก็บผลรวมและค่าจำนวนค่าที่ใช้วัดผล (Waiting Time, Turnaround Time, Throughput, Resource Wait Time) ตามหัวข้อ 8 ของโจทย์

```
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * รวบรวมและคำนวณค่าที่ใช้วัดผลของการรันหนึ่งครั้ง (หัวข้อ 8)
 *
 * ตัวเลขเดียวที่หลาย Worker แก่พร้อมกันจริง ๆ คือ "จำนวนงานที่เสร็จแล้ว"
 * จึงใช้ AtomicInteger เก็บ ส่วนเวลาต่าง ๆ ของแต่ละ Job (waiting, turnaround,
 * resource wait) เก็บอยู่ที่ตัว Job เองอยู่แล้ว (ผ่าน get/set ที่ Job.java
 * ทำให้ได้) printSummary() จึงแค่ไล่อ่านจาก allJobs ที่ Main ส่งมาตอนจบ
 */
public class Statistics {

    private final AtomicInteger completed = new AtomicInteger(0);

    /** บันทึกว่างานชิ้นหนึ่งเสร็จแล้ว เรียกโดย Worker หลายตัวพร้อมกันได้ */
    public void recordCompletion(Job job) {
        completed.incrementAndGet();
    }

    /** จำนวนงานที่เสร็จแล้ว ใช้โดย Monitor และใช้ตรวจว่างานครบหรือยัง */
    public int completedCount() {
        return completed.get();
    }

    /**
     * พิมพ์ตารางสรุปผลตอนจบโปรแกรม
     * avg WT, avg TAT, avg Resource Wait เป็นจำนวนเต็ม ms
     * Throughput อย่างน้อย 2 ตำแหน่งทศนิยม
     */
    public void printSummary(List<Job> allJobs, long makespanMs) {
        int jobCount = allJobs.size();

        long totalWaiting = 0;
        long totalTurnaround = 0;
        long totalResourceWait = 0;
        int resourceJobCount = 0;

        for (Job job : allJobs) {
            totalWaiting += job.waitingTimeMs();
            totalTurnaround += job.turnaroundTimeMs();
            if (job.resource != ResourceType.NONE) {
                totalResourceWait += job.getResourceWaitMs();
                resourceJobCount++;
            }
        }

        double avgWaitingMs = jobCount == 0 ? 0.0 : (double) totalWaiting / jobCount;
        double avgTurnaroundMs = jobCount == 0 ? 0.0 : (double) totalTurnaround / jobCount;
        double avgResourceWaitMs =
            resourceJobCount == 0 ? 0.0 : (double) totalResourceWait / resourceJobCount;

        double throughput = makespanMs <= 0 ? 0.0
            : completed.get() / (makespanMs / 1000.0);

        System.out.println();
        System.out.println("==== Summary =====");
        System.out.printf("Jobs completed   : %d/%d\n", completed.get(), jobCount);
        System.out.printf("Makespan       : %d ms\n", makespanMs);
        System.out.printf("Avg Waiting Time : %d ms\n", Math.round(avgWaitingMs));
        System.out.printf("Avg Turnaround Time: %d ms\n", Math.round(avgTurnaroundMs));
        System.out.printf("Throughput      : %.2f jobs/sec\n", throughput);
        System.out.printf("Avg Resource Wait : %d ms (over %d resource jobs)\n",
            Math.round(avgResourceWaitMs), resourceJobCount);
    }
}
```

}

คำอธิบายทีละส่วน

- `completed (AtomicInteger)` — ตัวนับจำนวนงานที่เสร็จแล้ว ใช้ `AtomicInteger` แทน `int` ธรรมดา เพราะ Worker หลายตัวอาจเรียก `recordCompletion()` พร้อมกัน ถ้าใช้ `int` ธรรมดากับ `completed++` การเพิ่มค่าจริง ๆ แล้วเป็น 3 ขั้นตอนซ้อนอยู่ (อ่านค่า, บวกหนึ่ง, เขียนค่ากลับ) ถ้าสอง Thread ทำ 3 ขั้นตอนนี้สลับกันพอดี อาจทำให้นับตกหล่นได้ `AtomicInteger.incrementAndGet()` รับประกันว่า ทั้ง 3 ขั้นตอนนี้เกิดขึ้นเป็นหนึ่งเดียวแบบแบ่งแยกไม่ได้ (atomic)
- `recordCompletion(job)` — แคเรียก `completed.incrementAndGet()` เท่านั้น ไม่ได้เก็บตัว `job` ไว้ซ้ำ เพราะข้อมูลของ Job แต่ละตัว (`waitingTimeMs`, `turnaroundTimeMs`, `resourceWaitMs`) เก็บอยู่ที่ตัว Job เองอยู่แล้ว
- `completedCount()` — คืนค่าปัจจุบันของตัวนับ ใช้โดย Monitor และใช้เช็คความเสร็จครบหรือยัง
- `printSummary(allJobs, makespanMs)` — วนลูป for ผ่านทุก Job ใน `allJobs` สะสมผลรวม 3 ค่า: `totalWaiting` (บวก `job.waitingTimeMs()` ทุกตัว), `totalTurnaround` (บวก `job.turnaroundTimeMs()` ทุกตัว), `totalResourceWait` (บวกเฉพาะตัวที่ `resource != NONE` เท่านั้น พร้อมนับ `resourceJobCount`)
- การหารเฉลี่ย — ใช้ตัวดำเนินการ ?: เช็คก่อนว่า `jobCount` หรือ `resourceJobCount` เป็น 0 หรือไม่ ถ้าเป็น 0 ให้ใช้ 0.0 แทน เพื่อกันโปรแกรม crash จากการหารด้วยศูนย์
- `throughput` — คำนวณจาก `completed.get()` หารด้วย (`makespanMs / 1000.0`) เพื่อแปลงหน่วยจาก ms เป็นวินาทีก่อนหาร ทำให้ได้หน่วยเป็น `jobs/second`
- `System.out.printf` — `%d` คือพิมพ์จำนวนเต็ม, `%.2f` คือพิมพ์ทศนิยม 2 ตำแหน่ง, `%n` คือขึ้นบรรทัดใหม่ (เหมือน `\n` แต่ปรับตาม OS ให้อัตโนมัติ) ใช้ `Math.round()` บัดเศษ WT/TAT/RW ให้เป็นจำนวนเต็ม ms ตามที่หัวข้อ 14 กำหนด

7. JobGenerator.java

Thread ที่ปล่อยงานเข้าสู่ระบบตามเวลา arrivalMs ของแต่ละ Job

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.concurrent.BlockingQueue;

/**
 * ปล่อยงานเข้าสู่ระบบตามเวลา arrivalMs ของแต่ละ Job
 *
 * ส่งงานไปยัง Scheduler ผ่าน arrivalQueue (BlockingQueue) ไม่แตะ ReadyQueue
 * โดยตรงเลย (โจทย์ห้าม) รายการงานที่ได้มาเรียงตามไฟล์ ไม่ใช่ตามเวลา
 * จึง sort สำเนาของ list ตาม arrivalMs ก่อนเริ่มปล่อย
 *
 * เมื่อปล่อยงานครบแล้ว ส่ง POISON_PILL เข้า arrivalQueue หนึ่งตัว
 * เพื่อบอก Scheduler ว่าจะไม่มีการเพิ่มเข้ามาอีก
 */
public class JobGenerator extends Thread {

    private final List<Job> jobsByArrival;
    private final BlockingQueue<Job> arrivalQueue;
    private final ProjectLogger logger;

    public JobGenerator(List<Job> jobs, BlockingQueue<Job> arrivalQueue, ProjectLogger logger) {
        super("generator");
        this.jobsByArrival = new ArrayList<>(jobs);
        this.jobsByArrival.sort(Comparator.comparingLong(j -> j.arrivalMs));
        this.arrivalQueue = arrivalQueue;
        this.logger = logger;
    }

    @Override
    public void run() {
        try {
            for (Job job : jobsByArrival) {
                long waitMs = job.arrivalMs - logger.now();
                if (waitMs > 0) {
                    Thread.sleep(waitMs);
                }

                job.setActualArrivalMs(logger.now());
                logger.jobArrived(job);
                arrivalQueue.put(job);
            }

            arrivalQueue.put(ReadyQueue.POISON_PILL);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

คำอธิบายทีละส่วน

- jobsByArrival — ก๊อปปี้ list ที่ได้รับมา (new ArrayList<>(jobs)) แล้ว sort ตาม arrivalMs ก่อนใช้งาน ก๊อปปี้ไว้เพื่อไม่ไปยุ่งกับ list ต้นฉบับที่ Main ถืออยู่ (list เดิมต้องคง ลำดับตามไฟล์ไว้ให้ Statistics ใช้ตอนสรุปผล)
- .sort(Comparator.comparingLong(j -> j.arrivalMs)) — เรียง job ตามเวลาที่ควรเข้าระบบ จากนั้นน้อยไปมาก (j -> j.arrivalMs คือ lambda expression บอกว่า "เอาค่า arrivalMs ของแต่ละ job มาเทียบกัน") List.sort เป็น stable sort จึงไม่สลับลำดับงานที่ arrivalMs เท่ากัน (ยังคงเรียงตาม ลำดับไฟล์เดิมต่อไป)
- arrivalQueue — ช่องทางส่งงานต่อให้ Scheduler (ไม่ส่งเข้า ReadyQueue ตรง ๆ เพราะโจทย์ห้าม)
- run() — ลูป for — สำหรับแต่ละ job ตามลำดับเวลา: คำนวณ waitMs = arrivalMs - logger.now() (เวลาที่ต้องรอก่อนปล่อยงานนี้) ถ้า waitMs มากกว่า 0 ถึงจะ sleep (เขັคก่อน เพราะถ้า arrivalMs น้อยกว่าหรือเท่ากับเวลาปัจจุบันแล้ว Thread.sleep(ค่าติดลบ) จะทำให้โปรแกรม throw exception ทันที)

- `job.setActualArrivalMs(logger.now())` — บันทึกเวลาที่ "เข้าสู่ระบบจริง" ซึ่งอาจช้ากว่า `arrivalMs` เล็กน้อย (Thread อาจถูกปลุกช้ากว่ากำหนดเสมอ ขึ้นกับ OS)
- `arrivalQueue.put(job)` — ส่งงานเข้าคิว ใช้ `put()` แทน `add()` เพื่อในอนาคตเปลี่ยนไปใช้คิวแบบ มีขอบเขตจำกัด (ตอนนี้ยังไม่ block เพราะคิวไม่มีขอบเขต)
- หลัง for loop จบ — ส่ง `ReadyQueue.POISON_PILL` เข้า `arrivalQueue` หนึ่งตัว บอก Scheduler ว่าปล่อยงานครบทุกชิ้นแล้ว
- `catch (InterruptedException e)` — ถ้าถูกสั่งหยุดกลางทาง ให้เรียก `Thread.currentThread().interrupt()` เพื่อ "คืนสถานะ interrupt" กลับไป (ธรรมเนียมมาตรฐานของ Java เวลา catch `InterruptedException` แล้วไม่ได้จัดการต่อ ไม่ควรกลืนทิ้งเฉย ๆ)

8. Scheduler.java

Thread ที่รับงานจาก JobGenerator แล้วส่งต่อเข้า ReadyQueue

```
import java.util.concurrent.BlockingQueue;

/**
 * รับงานจาก JobGenerator (ผ่าน arrivalQueue) แล้วจัดเข้า Ready Queue
 *
 * ไม่ต้องรู้เงื่อนไข FCFS/Priority เลย เพราะ ReadyQueue เป็นคนจัดลำดับ
 * ให้เองอยู่แล้วตอนสร้าง หน้าของ Scheduler คือรับงานมาแล้วส่งต่อ
 *
 * เมื่อเจอ POISON_PILL จาก arrivalQueue (แปลว่า JobGenerator ปล่อยงานครบแล้ว)
 * ต้องส่ง POISON_PILL ต่อให้ Worker "ทุกตัว" ผ่าน ReadyQueue เพราะ Worker
 * แต่ละตัวรอ take() อยู่คนละที่ ถ้าส่งแค่ตัวเดียวจะมีแค่ Worker ตัวเดียว
 * ที่หยุดได้ ตัวอื่นจะค้างรอตลอดไป
 */
public class Scheduler extends Thread {

    private final BlockingQueue<Job> arrivalQueue;
    private final ReadyQueue readyQueue;
    private final int workerCount;
    private final ProjectLogger logger;

    public Scheduler(BlockingQueue<Job> arrivalQueue, ReadyQueue readyQueue,
        int workerCount, ProjectLogger logger) {
        super("scheduler");
        this.arrivalQueue = arrivalQueue;
        this.readyQueue = readyQueue;
        this.workerCount = workerCount;
        this.logger = logger;
    }

    @Override
    public void run() {
        try {
            while (true) {
                Job job = arrivalQueue.take();

                if (ReadyQueue.isPoisonPill(job)) {
                    for (int i = 0; i < workerCount; i++) {
                        readyQueue.add(ReadyQueue.POISON_PILL);
                    }
                    break;
                }

                logger.systemEvent("job=" + job.id + " READY");
                readyQueue.add(job);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

คำอธิบายทีละส่วน

- arrivalQueue, readyQueue, workerCount, logger — ฟิลด์ที่เก็บไว้ใช้ใน run() workerCount ถูกเพิ่มเข้ามาเพราะจำเป็นต้องรู้ว่าต้องส่ง POISON_PILL กี่ตัวตอนปิดระบบ
- run() — while(true) loop — เรียก arrivalQueue.take() เข้าไปเรื่อย ๆ (block ถ้ายังไม่มีงาน) ทุกครั้งที่ได้ job มาหนึ่งตัว เช็คว่าเป็น POISON_PILL หรือไม่
- ถ้าเป็น POISON_PILL — แปลว่า JobGenerator ปล่อยงานครบแล้ว วนลูป for ตั้งแต่ 0 ถึง workerCount ใส่ POISON_PILL ลง readyQueue "หนึ่งตัวต่อหนึ่งรอบ" (เท่ากับจำนวน Worker พอดี) แล้ว break ออกจาก while loop เพื่อจบ Thread นี้

- ทำไมต้องส่งหลายตัว — Worker แต่ละตัวรอ take() อยู่คนละทีกัน ถ้าส่ง pill แค่ตัวเดียว จะมี Worker แค่ 1 ตัวที่หยิบได้แล้วหยุด ตัวอื่นจะรอ take() ค้างตลอดไป (โปรแกรมไม่ปิด)
- ถ้าไม่ใช่ POISON_PILL — เป็นงานจริง: log ว่างานนี้ READY แล้วส่งเข้า readyQueue.add(job) Scheduler ไม่ต้องรู้เลยว่า ReadyQueue จัดลำดับแบบ FCFS หรือ Priority เพราะ ReadyQueue จัดการ เรียงนั่นเองทั้งหมด
- catch (InterruptedException e) — รูปแบบเดียวกับ JobGenerator

9. Worker.java

Thread ที่ดึงงานจาก ReadyQueue มาทำงานเสร็จ ไฟล์นี้มีรายละเอียดเยอะที่สุดเพราะเป็นจุดที่ต้องประสานงานกับ ResourceManager, Statistics และกลไกการปิดระบบพร้อมกันหมด

```
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Thread ที่ดึงงานจาก Ready Queue ไปทำงานเสร็จ
 *
 * ลำดับการทำงานของ Job หนึ่งชิ้น (หัวข้อ 6):
 * 1. รับงาน บันทึกลงเวลาเริ่ม
 * 2. จำลองงานหลักด้วย Thread.sleep(workMs)
 * 3. ถ้ามี resource: บันทึกลงเวลาเริ่มรอ แล้ว acquire
 * 4. จำลองการถือครองด้วย Thread.sleep(resourceMs)
 * 5. release แล้วบันทึกเวลาจบ
 *
 * การหยุด: หยิบได้ POISON PILL จาก ReadyQueue แปลว่าไม่มีงานเหลือแล้ว ให้หยุด
 * การนับว่า "งานเสร็จจริงหรือยัง" ใช้ CountDownLatch แยกต่างหาก (นับทุกครั้ง
 * ที่ทำ Job เสร็จจริงหนึ่งชิ้น) เพราะ POISON PILL บอกแค่ "ไม่มีงานใหม่เข้ามา"
 * ไม่ได้บอกว่า "งานที่มีอยู่ตอนนี้เสร็จหมดแล้ว"
 *
 * runningJobs (ที่ Monitor ต้องอ่าน) เก็บเป็น static AtomicInteger ของคลาสนี้
 * เพราะ Worker ทุกตัวใช้ class เดียวกัน จึงแชร์ตัวนับได้เลยโดยไม่ต้องส่ง
 * object อื่นเพิ่ม
 */
public class Worker extends Thread {

    private static final AtomicInteger RUNNING = new AtomicInteger(0);

    /** จำนวน Job ที่กำลังทำอยู่ตอนนี้ (รวมทุก Worker) ใช้โดย Monitor */
    public static int runningCount() {
        return RUNNING.get();
    }

    private final ReadyQueue readyQueue;
    private final ResourceManager resources;
    private final Statistics statistics;
    private final ProjectLogger logger;
    private final CountDownLatch allJobsDone;

    public Worker(String name, ReadyQueue readyQueue, ResourceManager resources,
        Statistics statistics, ProjectLogger logger, CountDownLatch allJobsDone) {
        super(name);
        this.readyQueue = readyQueue;
        this.resources = resources;
        this.statistics = statistics;
        this.logger = logger;
        this.allJobsDone = allJobsDone;
    }

    @Override
    public void run() {
        try {
            while (true) {
                Job job = readyQueue.take();

                if (ReadyQueue.isPoisonPill(job)) {
                    break;
                }

                processJob(job);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    /** ทำงานหนึ่งชิ้นให้จบตามลำดับ 5 ขั้นตอนบน */
}
```

```

private void processJob(Job job) throws InterruptedException {
    RUNNING.incrementAndGet();
    try {
        // ขั้น 1
        job.setStartMs(logger.now());
        logger.jobStarted(job);

        // ขั้น 2 (ห้ามสลับกับขั้น 3)
        Thread.sleep(job.workMs);
        logger.workFinished(job);

        // ขั้น 3-4-5
        if (job.resource != ResourceType.NONE) {
            job.setResourceWaitStartMs(logger.now());
            logger.resourceWaitStarted(job);

            resources.acquire(job.resource);
            long waitedMs = logger.now() - job.getResourceWaitStartMs();
            job.setResourceWaitMs(waitedMs);
            logger.resourceAcquired(job, waitedMs);

            try {
                Thread.sleep(job.resourceMs);
            } finally {
                // release ต้องอยู่ใน finally เสมอ กัน permit ค้าง
                // ถ้าเกิด interrupt ระหว่างถือ resource อยู่
                resources.release(job.resource);
                logger.resourceReleased(job);
            }
        }

        job.setCompletionMs(logger.now());
        logger.jobCompleted(job);
        statistics.recordCompletion(job);
    } finally {
        RUNNING.decrementAndGet();
        // countDown ต้องเกิดไม่ว่า Job จะจบแบบสำเร็จหรือถูก interrupt
        // กลางทาง ไม่งั้น Main จะรอที่ latch.await() ค้างตลอดไป
        allJobsDone.countDown();
    }
}
}

```

คำอธิบายทีละส่วน

- **RUNNING** (static AtomicInteger) — ตัวนับจำนวน Job ที่กำลังถูกทำอยู่ตอนนี้ เป็น static แปลว่า Worker ทุก object (ทุก Thread) ใช้ตัวแปรตัวเดียวกันนี้ ร่วมกัน จึงไม่ต้องส่ง object กลางผ่าน constructor เพิ่ม
- **runningCount()** — static method ให้ Monitor เรียกอ่านค่า RUNNING จากที่ไหนก็ได้ด้วย Worker.runningCount()
- **allJobsDone** (CountDownLatch) — ส่งมาจาก Main ตอนสร้าง Worker ใช้บอกว่า "งานชิ้นนี้ ประมวลผลจบแล้ว" (ดูหัวข้อ 1 เรื่องทำไมต้องมีสองกลไก)
- **run()** — วนลูป take() จาก readyQueue ถ้าได้ POISON_PILL ให้ break (หยุด Thread) ถ้าไม่ใช่ให้เรียก processJob(job) ไปเรื่อย ๆ
- **processJob()** ขั้น 1 — RUNNING.incrementAndGet() ก่อน (ถือว่า "เริ่มทำงานนี้แล้ว") จากนั้น setStartMs(logger.now()) และ log jobStarted
- **processJob()** ขั้น 2 — Thread.sleep(job.workMs) จำลองเวลาทำงานหลัก แล้ว log workFinished (ห้ามสลับลำดับกับขั้น 3 เพราะโจทย์กำหนดไว้ตายตัว)
- **processJob()** ขั้น 3-5 (ถ้ามี resource) — setResourceWaitStartMs + log resourceWaitStarted ก่อน แล้วเรียก resources.acquire(job.resource) (ตรงนี้จะรอถ้า permit หมด) พอได้สิทธิ์แล้ว คำนวณ waitedMs = เวลาตอนนี้ - เวลาที่เริ่มรอ แล้ว setResourceWaitMs + log resourceAcquired
- **try/finally** ขั้นใน — Thread.sleep(job.resourceMs) จำลองการถือครอง resource อยู่ใน try ส่วน resources.release(...) กับ log resourceReleased อยู่ใน finally เพื่อรับประกันว่า release ถูกเรียกเสมอ ไม่ว่า sleep จะถูก interrupt หรือไม่ก็ตาม (จุดนี้คือจุดกัน permit ค้างตามที่โจทย์เตือนไว้)

- หลังจบ if resource — setCompletionMs, log jobCompleted, statistics.recordCompletion(job)
- finally ชั้นนอกสุดของ processJob() — RUNNING.decrementAndGet() (ถือว่า "เลิกทำงานนี้แล้ว") แล้ว allJobsDone.countDown() เสมอ ไม่ว่า Job จะจบแบบสำเร็จ หรือถูก interrupt กลางทาง เพื่อให้ Main ค้างรอที่ latch.await() ตลอดไป

10. Monitor.java

Thread ที่รายงานสถานะระบบ (ready, running, completed, resource) เป็นระยะๆ ทุกประมาณ 1 วินาที

```
/**
 * Thread ที่รายงานสถานะระบบเป็นระยะ (หัวข้อ 10)
 *
 * รายงานทุกประมาณ 1,000 ms: ready, running, completed, สถานะ resource
 * ทุกค่าที่อ่านมาจากโครงสร้าง thread-safe อยู่แล้ว (BlockingQueue.size(),
 * AtomicInteger, Semaphore) จึงไม่ต้องใส่ synchronized เอง
 *
 * ตัวนับ running อยู่ที่ Worker.runningCount() (static) จึงไม่ต้องส่ง
 * object อะไรเพิ่มเข้ามาทาง constructor
 *
 * การหยุด: ไม่ได้อ่านจากคิวงาน จึงไม่ใช่ poison pill แต่ให้ Main เรียก
 * monitor.interrupt() ตอน shutdown ซึ่งจะไปสะดุดที่ Thread.sleep(1000)
 * * แล้วโยน InterruptedException ออกมาให้ลูบจับทันที
 */
public class Monitor extends Thread {

    private static final long REPORT_INTERVAL_MS = 1000L;

    private final ReadyQueue readyQueue;
    private final ResourceManager resources;
    private final Statistics statistics;
    private final ProjectLogger logger;

    public Monitor(ReadyQueue readyQueue, ResourceManager resources,
        Statistics statistics, ProjectLogger logger) {
        super("monitor");
        this.readyQueue = readyQueue;
        this.resources = resources;
        this.statistics = statistics;
        this.logger = logger;
    }

    @Override
    public void run() {
        try {
            while (!Thread.currentThread().isInterrupted()) {
                Thread.sleep(REPORT_INTERVAL_MS);

                int ready = readyQueue.size();
                int running = Worker.runningCount();
                int completed = statistics.completedCount();
                String resourceStatus = resources.status();

                logger.monitor(ready, running, completed, resourceStatus);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

คำอธิบายที่ละส่วน

- REPORT_INTERVAL_MS = 1000 — ค่าคงที่ที่กำหนดว่ารายงานทุกกี่ ms ประกาศเป็นค่าคงที่แยก เพื่อไม่ต้องเขียนเลข 1000 ลอย ๆ ซ้ำในโค้ด (magic number)
- constructor — รับ readyQueue, resources, statistics, logger เหมือนที่โจทย์กำหนดไว้ ไม่ต้องเพิ่ม parameter ใด ๆ เลย เพราะตัวนับ running ไปอยู่ที่ Worker.runningCount() (static) แทนที่จะส่งผ่าน constructor
- run() — while loop — เงื่อนไขคือ !Thread.currentThread().isInterrupted() ในแต่ละรอบ: Thread.sleep(1000) ก่อน แล้วค่อยอ่านค่าทั้ง 4 (ready, running, completed, resourceStatus) แล้วพิมพ์ผ่าน logger.monitor(...)
- ทำไม sleep ก่อนแล้วค่อยอ่านค่า — ทำให้รายงานแรกเกิดที่ประมาณ 1 วินาทีหลังเริ่ม ไม่ใช่ ทันทีที่ t=0 ซึ่งตรงกับที่โจทย์บอกว่า "ไม่ต้องแม่นยำตรง 1000ms ทุกครั้ง" อยู่แล้ว

- ทำไมไม่ต้องป้องกันการอ่านเพิ่ม — `readyQueue.size()`, `Worker.runningCount()`, `statistics.completedCount()` ล้วนอ่านจากโครงสร้างข้อมูลที่เป็น thread-safe อยู่แล้วในตัวเอง (`BlockingQueue`, `AtomicInteger`)
การอ่านค่าแต่ละตัวจึงปลอดภัยโดยไม่ต้องใส่ `synchronized` เอง
- การหยุด Monitor — ไม่ได้ใช้ `POISON_PILL` แบบ Worker เพราะ Monitor ไม่ได้อ่านจากคิวงาน แต่ให้ Main เรียก `monitor.interrupt()` ตอน shutdown ซึ่งจะไปสะดุดตรง `Thread.sleep(1000)` แล้วโยน `InterruptedException` ออกมาทันที ทำให้ while loop จบตัวเอง